

A deep learning algorithm to accelerate algebraic multigrid methods in finite element solvers of 3D elliptic PDEs [☆]

Matteo Caldana ^{*}, Paola F. Antonietti, Luca Dede^{*}

MOX, Dipartimento di Matematica, Politecnico di Milano, Piazza Leonardo da Vinci 32, Milano, 20133, Italy

ARTICLE INFO

Keywords:

Algebraic multigrid methods
Partial differential equations
Finite element method
Elliptic problems
Deep learning
Convolutional neural networks

ABSTRACT

Algebraic multigrid (AMG) methods are among the most efficient solvers for linear systems of equations and they are widely used for the solution of problems stemming from the discretization of Partial Differential Equations (PDEs). A severe limitation of AMG methods is the dependence on parameters that require to be fine-tuned. In particular, the strong threshold parameter is the most relevant since it stands at the basis of the construction of successively coarser grids needed by the AMG methods. We introduce a novel deep learning algorithm that minimizes the computational cost of the AMG method when used as a finite element solver. We show that our algorithm requires minimal changes to any existing code. The proposed Artificial Neural Network (ANN) tunes the value of the strong threshold parameter by interpreting the sparse matrix of the linear system as a gray scale image and exploiting a pooling operator to transform it into a small multi-channel image. We experimentally prove that the pooling successfully reduces the computational cost of processing a large sparse matrix and preserves the features needed for the regression task at hand. We train the proposed algorithm on a large dataset containing problems with a strongly heterogeneous diffusion coefficient defined in different three-dimensional geometries and discretized with unstructured grids and linear elasticity problems with a strongly heterogeneous Young's modulus. When tested on problems with coefficients or geometries not present in the training dataset, our approach reduces the computational time by up to 30%.

1. Introduction

Geometric multigrid methods [1,2] are among the state-of-the-art solvers for large linear systems that come from the discretization of Partial Differential Equations (PDEs). They are applicable to a wide range of discretizations such as polyhedral discontinuous Galerkin [3,4] and virtual elements methods [5,6]. However, their limitation is that they rely on a sequence of coarser grids that must be available a-priori to solve the problem. The algebraic multigrid (AMG) methods are a highly scalable [7] generalization that is capable of building this hierarchy of grids algebraically. A challenge behind the algebraic construction

of a coarser mesh is the selection of the prolongation operator, which in turns relies on seeing the sparse matrix of the linear system as a weighted graph and defining a strong threshold parameter to partition the aforementioned graph.

The AMG method was first introduced in the 1980s in [8] and following papers [9–11], gaining momentum in the following years, see e.g. [12–15]. In recent years, modifications have been proposed to the AMG to improve its efficiency, like smoothed aggregation [16–18] and extend its range of application to different discretization techniques, like Ritz-type finite element methods [19–21], discontinuous Galerkin [22–24], and hybrid discretizations [25,26], and to different physics

[☆] Funding: Funded by the European Union (ERC, NEMESIS, project number 101115663). Views and opinions expressed are however those of the authors only and do not necessarily reflect those of the European Union or the European Research Council Executive Agency. Neither the European Union nor the granting authority can be held responsible for them.

P.F.A. and L.D. have been partially funded by the research project PRIN 2020 (n. 20204LN5N5) funded by Italian Ministry of University and Research (MUR).

L.D. acknowledges the support by the FAIR (Future Artificial Intelligence Research) project, funded by the NextGenerationEU program within the PNRR-PE-AI scheme (M4C2, investment 1.3, line on Artificial Intelligence), Italy.

The authors are members of the INdAM Research group GNCS.

The present research is part of the activities of “Dipartimento di Eccellenza 2023–2027”.

^{*} Corresponding author.

E-mail addresses: matteo.caldana@polimi.it (M. Caldana), paola.antonietti@polimi.it (P.F. Antonietti), luca.dede@polimi.it (L. Dede^{*}).

equations like Navier-Stokes equations [27,28], Maxwell's equations [29,30], linear elasticity [31,32], magnetohydrodynamics [33], and poromechanics [34,35]. There is also a wide literature that tackles the AMG from a theoretical point of view, the foundation was laid down in [36–39] and the most recent survey is found in [40]. In particular, for the two-level convergence theory, one of the first results can be found in [8,11]. Extensions to variational problems are first presented in [41]. Sharper results, including two-sided bounds, are contained in [42] and also considered by [43,44]. These two-level results leverage the abstract theory presented in [36,38]. A survey of these and related findings is available in [45].

In particular, we consider the finite element (FE) [46] discretization of diffusion and linear elasticity problems with strongly heterogeneous coefficients in three-dimensional space (3D). These problems represent a challenging benchmark since they feature a large number of unknowns and a very ill-conditioned system matrix. Consequently, the efficient application of the AMG method is the key to obtain fast convergence.

In this work, we propose to use Deep Learning (DL) [47] techniques to find the optimal value of the strong threshold parameter, depending on the problem we want to solve. In particular, Artificial Neural Networks (ANNs) have gained widespread popularity for a variety of applications. They are versatile tools that are progressively being used in scientific computing [48–50], particularly in the numerical approximation of PDEs and model order reduction [51,52]. ANNs can also be utilized within a data-driven framework to provide alternative closure models, which involve learning input-output relationships in complex physical processes [53–55]. Since we treat the sparse matrix of the linear system like an image, part of the ANN that we employ is comprised by a Convolutional Neural Network (CNN). CNNs are among the most applied neural architectures to analyze visual imagery and achieved breakthrough results [56–60]. Today, they are only surpassed by visual transformer [61], which however have a much more complex architecture and are much more difficult to optimize. Moreover, CNNs have already been successfully applied in the field of scientific computing [62,63].

The combination of finite element/multigrids methods and machine learning has already been used in literature. For instance, in [64,65] there is the first attempt of optimizing the multigrid parameters, in [66–68] DL is used to perform grid refinement and agglomeration, in [69,70] Graph Neural Networks (GNNs) are used to learn the prolongation operator and in [71] reinforcement learning is used to perform graph coarsening, in [72,73] DL is used to enhance domain decomposition methods. In this paper, we propose a DL-based algorithm that is able to tune on-the-fly the strong threshold parameter so as to minimize the computational time needed to solve the linear system at hand. The main difference with the previous works is that our algorithm is completely non intrusive: following the approach described in [74], it does not require any change to existing code (neither to the FE nor the AMG solver). This guarantees a wider range of applicability and means that we can rely on all the classical theoretical results regarding convergence. Namely, an ANN is trained to predict the computational cost of solving a linear system given as input a certain value of the strong threshold parameter and a small multi channel image representing the sparse matrix of the linear system. This representation is built by employing the pooling operator used in CNN. However, this algorithm struggles to effectively manage the increased complexity that arises when dealing with 3D problems. In this work we introduce a set of improvements that make the algorithm fast and accurate also for three-dimensional test cases. More precisely, we propose a pre-processing step that assures data quality, we enhance the architecture of the ANN by adding an additional output variable measuring the confidence of the neural network on the prediction and we improve the pooling step by means of a four channel tensor. We also show how to speed-up the training by employing layer freezing. We found that the proposed ANN-enhanced AMG method reduces significantly the computational cost

Algorithm 1 One Iteration of the V-cycle of the AMG method $\mathbf{u}^{(k)} = \text{vcycle}^k(\mathbf{u}^{(k)}, \mathbf{f}^{(k)}, \{(A^{(j)}, B_1^{(j)}, B_2^{(j)})\}_{j=k}^M, \{(I_j^{j+1}, I_{j+1}^j)\}_{j=k}^{M-1}, v_1, v_2)$.

```

1: if  $k = M$  then
2:    $\mathbf{u}^{(M)} = \text{gaussian\_elimination}(A^{(M)}, \mathbf{f}^{(M)})$ 
3: else
4:    $\mathbf{u}^{(k)} \leftarrow \text{smooth}^{v_1}(A^{(k)}, B_1^{(k)}, \mathbf{u}^{(k)}, \mathbf{f}^{(k)})$ 
5:    $\mathbf{r}^{(k+1)} \leftarrow I_k^{k+1}(\mathbf{f}^{(k)} - A^{(k)}\mathbf{u}^{(k)})$ 
6:    $\mathbf{e}^{(k+1)} \leftarrow \text{vcycle}^{k+1}(\mathbf{u}^{(k)}, \mathbf{f}^{(k)}, \{(A^{(j)}, B_1^{(j)}, B_2^{(j)})\}_{j=k+1}^M, \{(I_j^{j+1}, I_{j+1}^j)\}_{j=k+1}^{M-1}, v_1, v_2)$ 
7:    $\mathbf{u}^{(k)} \leftarrow \mathbf{u}^{(k)} + I_{k+1}^k \mathbf{e}^{(k+1)}$ 
8:    $\mathbf{u}^{(k)} \leftarrow \text{smooth}^{v_2}(A^{(k)}, B_2^{(k)}, \mathbf{u}^{(k)}, \mathbf{f}^{(k)})$ 
9: end if

```

(elapsed time) needed to solve the linear system compared to employing the pre-defined choice of the parameters based on trial-and-error, experience, and literature. Our work is closely tied to adaptive AMG, denoted as α AMG, initially introduced in [75]. The α AMG methods aim to determine optimal interpolation patterns through a limited number of smoothing iterations. Despite the added computational overhead of these iterations, they lead to more effective error reduction across multiple levels. This approach resembles our methodology: the cost of the forward pass in ANN to tune the strong threshold parameter, subsequently influencing the construction of the interpolation operator, balances out with a reduction in the computational cost of the AMG method. Moreover, in [75], α AMG is compared with calibrated AMG, a manually fine-tuned variant of the AMG method. The authors' findings indicate that while α AMG outperforms standard AMG, its costs are notably higher compared to calibrated AMG. However, this is achieved by using a significant amount of time to manually tune the parameters of the calibrated AMG. Our aim is to exploit the advantages of both approaches: finely tuning the AMG method at minimal expense by leveraging the fast online evaluation of ANNs.

This work is organized as follows. Section 2 introduces the mathematical framework of the AMG methods, in particular we define and show the importance of the strong threshold parameter. In Section 3, we present our algorithm. We describe the architecture of the ANN and show how to apply the pooling operator to a large sparse matrix. Section 4 is devoted to the numerical validation on our method. We first make preliminary sensitivity analysis of the hyperparameters of the ANN. We then apply our algorithm to a family of elliptic problems with a strongly heterogeneous diffusion coefficient on structured and unstructured meshes and to a family of linear elasticity problems with a strongly heterogeneous Young's modulus. Finally, in Section 5, we draw our conclusions with future developments.

2. AMG methods

To solve the linear $n_1 \times n_1$ symmetric positive definite (SPD) system $\mathbf{A}\mathbf{u} = \mathbf{f}$ the AMG [8,11,40] makes use of the following ingredients. A smoother, denoted $\text{smooth}^v(\mathbf{A}, \mathbf{B}, \mathbf{u}_0, \mathbf{f})$, defined as v applications of a linear iterative method $\mathbf{u}_{l+1} = \mathbf{u}_l + \mathbf{B}(\mathbf{f} - \mathbf{A}\mathbf{u}_l)$, $l \geq 0$, where $\mathbf{B} \in \mathbb{R}^{n_1 \times n_1}$ defines the iteration matrix and $\mathbf{u}_0$ is the initial guess. A sequence of interpolation operators $I_k^{k-1} \in \mathbb{R}^{n_{k-1} \times n_k}$ and restriction operators $I_{k-1}^k \in \mathbb{R}^{n_k \times n_{k-1}}$ for $k = 2, \dots, M$, such that

$$I_k^{k+1} = (I_{k+1}^k)^T, A^{(k+1)} = I_k^{k+1} A^{(k)} I_{k+1}^k, \forall k = 1, \dots, M-1, A^{(1)} = A. \quad (1)$$

The V-cycle multigrid iteration is defined in Algorithm 1, where $B_1^{(k)}, B_2^{(k)}$ are a sequence of pre- and post-smoothers for $k = 1, \dots, M$ with $n_1 > n_2 > \dots > n_M$, and v_1, v_2 are the number of pre- and post-smoothing iterations. The V-cycle relies on the fact that the error is more efficiently reduced by first projecting the residual equation $\mathbf{A}\mathbf{e} = \mathbf{r} = \mathbf{f} - \mathbf{A}\mathbf{u}_v$ on a coarser space, and then interpolating the solution back to the original space to apply the correction $\mathbf{u}_v + \mathbf{e}$.

2.1. Interpolation operator and coarse-grid selection

Interpolation operators are built by splitting the variables into two sets, the one that needs interpolation and the one that are kept at the coarse level [11]:

$$(I_{k+1}^k \mathbf{x})_i = \begin{cases} (\mathbf{x})_i & \text{if } i \in C^k, \\ \sum_{j \in \mathcal{F}_i^k} \omega_{ij}^k (\mathbf{x})_j & \text{if } i \in \mathcal{F}^k, \end{cases} \quad (2)$$

where $\mathbf{x} \in \mathbb{R}^{n_{k+1}}$ is a generic vector with entries $(\mathbf{x})_i$, $C^k/\mathcal{F}^k$ is a coarse/fine partition of the set $\mathcal{N}_k = \{1, \dots, n_k\}$, $C_i^k = \{j \in C^k : a_{ij} \neq 0\}$ is a subset of C^k that depends on i , ω_{ij}^k is a set of weights and a_{ij} are the entries of A . In the following, we exclusively consider the case of two levels. Hence, we will omit the subscript and superscript k .

To build ω_{ij} we have to define when the unknown $(\mathbf{u})_i$ can be interpolated from $(\mathbf{u})_j$, that is to define when the variable i strongly depends of j .

Definition 1 ([11]). Given a threshold parameter $0 < \theta \leq 1$, the set of variables on which the variable i strongly depends on is

$$S_i = \{j \neq i : -a_{ij} \geq \theta \max_{l \neq i} \{-a_{il}\}, j = 1, \dots, n_k\}.$$

Hence, the set of variables j influenced by i is $S_i^\top = \{j : i \in S_j, j = 1, \dots, n_k\}$. The underlying assumption of AMG (see [11]) is that the error satisfies

$$\sum_{j \in C_i} a_{ij}(\mathbf{e})_j + \sum_{j \in D_i^s} a_{ij}(\mathbf{e})_j + \sum_{j \in D_i^w} a_{ij}(\mathbf{e})_j = 0, \quad \forall i = 1, \dots, n,$$

where $D_i^s = \mathcal{F} \cap S_i$ and $D_i^w = \{j \in \mathcal{N} : a_{ij} \neq 0, j \notin S_i\}$. This yields the formula for the weights (where $\hat{a}_{ij} = a_{ij}$ if $a_{ij}a_{ii} \leq 0$ and zero otherwise)

$$\omega_{ij} = -\frac{1}{a_{ij} + \sum_{l \in D_i^w} a_{il}} \left(a_{ij} + \sum_{l \in D_i^s} \frac{a_{il}\hat{a}_{lj}}{\sum_{m \in C_i} \hat{a}_{lm}} \right) \quad (3)$$

Among the several techniques used to perform the $C/\mathcal{F}$ splitting, we outline the CLJP (Cleary-Luby-Jones-Plassman) algorithm [76]. Let $\mathcal{G}$ be the graph of variable dependencies $\mathcal{G} = (\mathcal{N}, \mathcal{E})$ with edges $\mathcal{E} = \{(i, j) \in \mathcal{N} \times \mathcal{N} : j \in S_i\}$. Then, until all vertexes are flagged, we repeat the following procedure:

1. Let $\eta(i) = |S_i^\top| + \tilde{\eta}$, $\tilde{\eta}$ random number in $(0, 1)$. If $\eta(i) < 1$, flag i as $\mathcal{F}$.
2. For each $i \in D = \{i \in V : \eta(i) > \eta(j) \forall j \in S_i \cap S_i^\top\}$ (independent set)
 - 2.1. For each $j \in S_i$, update $\eta(j)$ and remove the edge (i, j) from $\mathcal{E}$.
 - 2.2. For each $j \in S_i^\top$, remove (j, i) from $\mathcal{E}$
 - 2.2.1. For each $k \in S_j^\top \cap S_i^\top$, update $\eta(j)$ and remove (k, j) from $\mathcal{E}$.
3. Every point in D is flagged as C .

This algorithm is applied at each level until the size of the grid operator n_k is smaller than a certain given value n_{min} , that we fix equal to two. Thus, given the strong threshold θ we are able to construct the succession of operators needed for the V-cycle. Algorithm 2 showcases the full AMG method.

3. ANN-enhanced AMG

We now introduce the ideas behind the proposed AMG-ANN algorithm. We have shown that the strong threshold parameter θ heavily conditions the construction of the interpolation operator that stands at the basis of the AMG method. In literature, this value is usually fixed to 0.25 and 0.5 when applying the AMG to the solution of linear systems

Algorithm 2 AMG algorithm $\mathbf{u} = \text{AMG}(\mathbf{u}, A, \mathbf{f}, \theta, \{(B_1^{(j)}, B_2^{(j)})\}_{j=k}^M, N_{max}, tol)$.

```

1: build  $\{C^k, \mathcal{F}^k\}_{k=1}^M$  using  $\theta$  by means of CLJP
2: build the operators  $\{I_{j+1}^j\}_{j=k}^{M-1}$  employing Eq. (2) and Eq. (3)
3: build the operators  $\{I_j^{j+1}\}_{j=k}^{M-1}$  and  $\{(A^{(j)})\}_{j=k}^M$  by means of Eq. (1)
4: while  $k < N_{max}$  and  $\|\mathbf{A}\mathbf{u} - \mathbf{f}\| / \|\mathbf{f}\| < tol$  do
5:    $\mathbf{u} \leftarrow \text{vcycle}^1(\mathbf{u}, \mathbf{f}, \{(A^{(j)}, B_1^{(j)}, B_2^{(j)})\}_{j=k}^M, \{(I_j^{j+1}, I_{j+1}^j)\}_{j=k}^{M-1}, v_1, v_2)$ 
6:    $k \leftarrow k + 1$ 
7: end while

```

that stem from the discretization of 2D and 3D PDEs, respectively. Our algorithm aims at making an accurate choice of θ so as to minimize the computational cost of the AMG method. Cost can be minimized with respect to various measures, however we choose θ for two reasons. First, it is the most commonly used parameter, present in almost all the AMG algorithms. Second, from our experiments, it seems that among all the parameters we tested, it has one of the largest influences on the computational cost. Other factors such as the choice of the smoother and the number of smoothing steps also significantly contribute to overall computational cost, however, they are most relevant when these two parameters are optimized concurrently. We point out that, in order to keep low the computational costs involved in the optimization step and to better analyze the proposed algorithm, we decide not to optimize multiple parameters of the AMG at the same time. Indeed, when optimizing for more than one parameter we would expect to find many local minima and this would hinder the study of the generalization capability of the neural network. For these reasons, in this work, we focus only on the strong threshold parameter. Future research efforts are already directed towards adding the ability to choose the best combination among many parameters, such as the type of smoother and the number of smoothing steps.

To this end, we devise an algorithm that exploits DL techniques to predict the value of θ that minimizes the elapsed CPU time needed to solve the given linear system $\mathbf{A}\mathbf{u} = \mathbf{f}$. Namely, we propose to use an ANN $\mathcal{F}$ to predict the computational time t , that is the number of seconds needed to solve a linear system with the AMG method (including the setup phase), and its confidence of that prediction σ^2 , that is the square of the committed error, given as input a small multi-channel image V extracted from the matrix A , the FE degree p , the logarithm of the size of A and the value of the strong threshold parameter θ . Then, online, we perform a 1D optimization to find the optimal θ^* that minimizes the predicted cost $\tilde{t}$ (we use the tilde to denote quantities approximated by the ANN). The architecture of the ANN is shown in Fig. 1.

This section is structured as follows: in Section 3.1, we briefly review how we collect data for the training of the ANN; in Section 3.2 and Section 3.3 we explain how to compress the large sparse matrix A so that can be used as input of a neural network; Section 3.4 details the architecture of the ANN and how to predict the optimal value of θ . Finally, in Section 3.5 we collect together the results of the previous section and describe the AMG-ANN algorithm. To make the article self-contained we discuss the basics of DL in Appendix A.

3.1. Gathering and smoothing data

In this section we describe how we collect the data used to train the neural network. Namely, we are interested in t , the sum of the time to setup the AMG method and the execution time to convergence. Since the measurements of t are affected by errors, we need to repeat the measurement. However, this drives up the cost of the method. We would prefer, instead, to use a performance model for predicting the time. We tested several multivariate polynomial models that take into account n_k , nnz_k (the size and number of non-zero of $A^{(k)}$ at each level k , respectively) and the approximate convergence factor $\rho = (\mathbf{r}^{(N_{it})}/\mathbf{r}^{(0)})^{N_{it}}$ (where N_{it} is the number of iterations needed to reach convergence

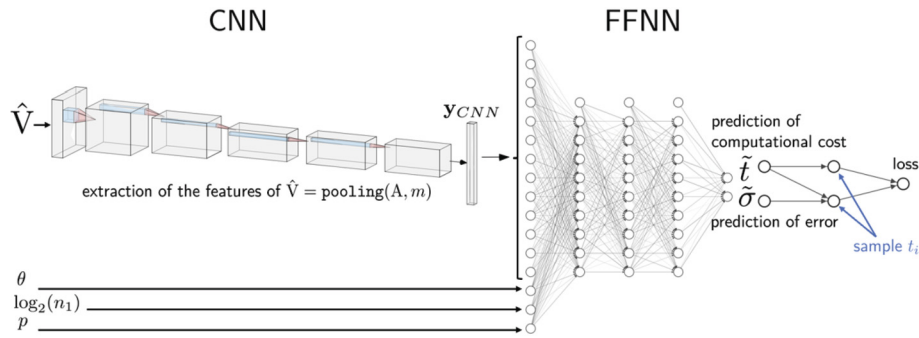


Fig. 1. Architecture of the proposed ANN used to predict the optimal value of the strong threshold parameter θ .

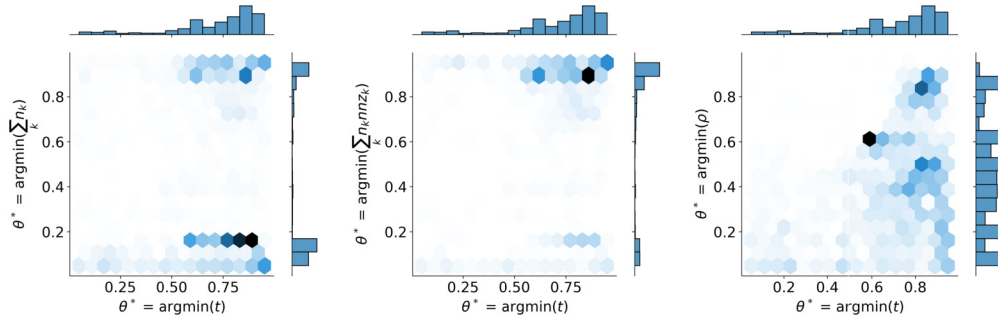


Fig. 2. Joint plot of the optimal strong threshold chosen with respect to different metrics. On the abscissas we have the θ^* that minimizes the total execution time t . None of the performance models considered produces a θ^* that correlates with it.

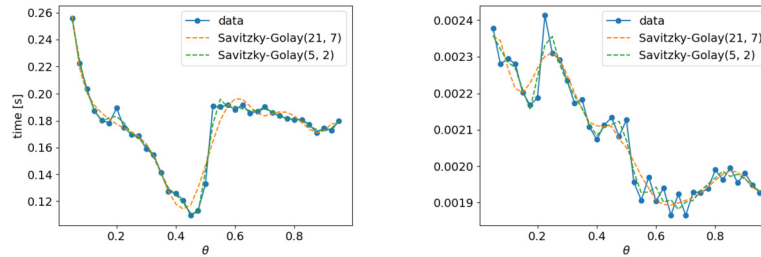


Fig. 3. Examples of smoothed data. Left: the smoother with default parameters moves the position of the minima, in this case a filter with window size five and order two better captures the behavior of the data. Right: the filter successfully smooths spurious oscillations due to the noise in the measurements.

and $\mathbf{r}^{(j)}$ is the residual at the j -th iteration), but none were able to consistently give the same choice of θ that we have by minimizing t . We report some relevant tests in a joint plot in Fig. 2: we can see that the two quantities do not show any correlation. Our explanation is that there is a complex interaction of factors that drives the total time t . Among these factors there is the setup time and the trade-off between performing a larger number of cheaper cycles and a lower number of more expensive cycles.

Then, to minimize the cost of collecting the data we employ the following procedure: we repeat the measurements r times with r between 2 and 100, where r is chosen inversely proportional to the mean elapsed time in the first two measurements. Indeed, the uncertainty in the measurements is caused by the tasks scheduled by the operating system in concurrence with the AMG solver that perturb the CPU load, hence if t is larger the uncertainty is smaller since these perturbations average out. This hypothesis is supported by our measurements, indeed we have observed that the sample variance of our measurements is inversely proportional to t (fixed r). The elapsed CPU time t is then defined as the mean of the measurements.

Moreover, we regularize data using Savitzky-Golay filter [77]. Namely, we fit a polynomial to a successive sub-sets of adjacent data

points (called window) by means of least squares. We use a window of size 21 and polynomial degree 7 for uniformly sampled values of θ . The two parameters were selected through a process of manual tuning, whereby various combinations were tested and assessed for their performance on a subset of problems within the dataset. The selection was guided by visual inspection of the resulting fits, with consideration given to balancing model complexity and accuracy. To further validate the result of the filtering we have checked that the filter maintains the positive sign of the data and we manually reviewed the cases where the difference between the minimum of the filtered and unfiltered (raw) data is larger than a certain threshold. In these cases we checked if it would be appropriate to increase the degree of the polynomial or change the size of the window. Indeed, the smoothing sometimes changes the position of the minima in sharp valleys since it tends to round out sharp features in the data, potentially shifting the position of minima within these valleys. Indeed, a suboptimal selection of parameters for the smoothing operator may substantially degrade the accuracy of the predictions of our algorithm. Our experiments show that appropriately calibrated smoothing yields improvements in accuracy amounting to a few percentage points. Thus, the choice to forego

smoothing remains a better option compared to employing an untuned smoothing operator. Fig. 3 showcases an example.

Computing time also depend on “external” parameters, such as the machine used for the computation. To overcome the variability due to the hardware and, at the same time, improve the training, we normalize the data between zero and one, independently for each subset of outputs in our dataset defined by fixing a matrix A . In this way, we require only that the computations concerning the same A are carried on the same machine. Once the multigrid solver is used in parallel, another external factor that may influence the computational cost is the number of CPU cores. We can easily treat this variable as another parameter of the AMG method, however, as mentioned before, this extension is left for future works. The computations were carried out (in serial) on various nodes of the HPC cluster at MOX with processor Intel® Xeon® Gold 6238R.

3.2. Applying the pooling operation to large sparse matrices

The first step of our algorithm is to transform the large sparse matrix A into a small tensor (that can be seen as a multi-channel image) so that it can be cheaply processed by the ANN. Drawing on the well-established benefits of pooling, which include translation invariance, computational efficiency, and hierarchical representation (see Appendix A for details), we aim at extending its application to the sparse matrix A . Indeed, A can be seen as a large gray scale image and we want to apply the pooling operation (before the training) to reduce the computational cost associated with handling such a large amount of data, and to prune unimportant features. Our hypothesis is that not all the information contained in A is needed for the regression problem at hand. We call $V = \text{pooling}(A, m) \in \mathbb{R}^{m \times m \times 4}$ the result of the pooling of A , where $m \in \mathbb{N}$ is a hyperparameter of the ANN that defines the size of V and is defined as follows, let $q = \lceil n_1/m \rceil$

$$\begin{aligned} v_{ij1} &= \max_{i_1, j_1=1, \dots, q} \{ \max\{0, \tilde{a}_{iq+i_1, jq+j_1}\} \}, \\ v_{ij2} &= \max_{i_1, j_1=1, \dots, q} \{ \max\{0, -\tilde{a}_{iq+i_1, jq+j_1}\} \}, \\ v_{ij3} &= \sum_{i_1=1}^q \sum_{j_1=1}^q \tilde{a}_{iq+i_1, jq+j_1}, \quad v_{ij4} = \sum_{i_1=1}^q \sum_{j_1=1}^q \chi_{(0, +\infty)}(\tilde{a}_{iq+i_1, jq+j_1}), \end{aligned}$$

where χ is the indicator function and $\tilde{a}_{ij} = a_{ij} \chi_{i \leq n_1, j \leq n_1}$. We report a possible implementation in Algorithm 3. Here, we are assuming that the sparse matrix A is stored in coordinate list format, however the algorithm can be easily generalized to other sparse storage formats. The only difference with the pooling operation usually applied in CNNs is that instead of extracting the maximum, we extract the following features (always in a rectangular neighborhood): maximum of the positive part, maximum of the negative part, the sum, the number of non-zeros (nnz) entries. The insight behind extracting these features is the following: the positive part, the negative part and the sum are relevant in the definition of the weights of the interpolation operator Eq. (3), the nnz count is an indicator of the sparsity of the neighborhood. Adding other features could be helpful, but we found this to be a good compromise between accuracy and computational cost. Moreover, these feature preserve the sparsity of the matrix, that is, if we consider a rectangular neighborhood of A of only zeros the result will be zero. The algorithm has low complexity, namely $O(nnz)$, that in the case of FE means $O(p \cdot n_1)$. Moreover, we have supported experimentally that its elapsed CPU time is negligible with respect to the one to solve the linear system. This is a necessary condition for this algorithm to be worthwhile. We remark that Algorithm 3 could be easily generalized to work in parallel.

3.3. Normalization

Still V , defined as before, cannot be directly used as input of an ANN. Normalization of data is a necessary step to assure fast convergence in neural networks. To this end we employ the following logarithmic

Algorithm 3 Pooling algorithm $V = \text{pooling}(A, m)$.

```

1: access  $A$  in COO form and extract its: size  $n_1$ , val, row, col
2: initialize  $V$  to an  $m \times m \times 4$  dense tensor with all zero entries  $v_{ijl} = 0$ 
3:  $q \leftarrow n_1/m$ ,  $p \leftarrow n_1 \bmod m$ ,  $t \leftarrow (q+1)p$ 
4: for  $k=0$  to  $\text{val.size}() - 1$  do
5:    $i \leftarrow \text{row}[k]/(q+1)$  if  $(\text{row}[k] - t)/(q+1) < p$ 
6:    $j \leftarrow \text{col}[k]/(q+1)$  if  $(\text{col}[k] - t)/(q+1) < p$  else  $(\text{col}[k] - t)/q + p$ 
7:    $v_{ij1} \leftarrow \max\{\max\{0, \text{val}[k]\}, v_{ij1}\}$ 
8:    $v_{ij2} \leftarrow \max\{\max\{0, -\text{val}[k]\}, v_{ij2}\}$ 
9:    $v_{ij3} \leftarrow v_{ij3} + \text{val}[k]$ 
10:   $v_{ij4} \leftarrow v_{ij4} + 1$ 
11: end for

```

normalization for each channel of the input V since it has been shown in [74] to outperform a classical linear normalization in this kind of applications:

$$\hat{v}_{ij} = \frac{\log(|v_{ij}| + 1)}{\max_{i,j} |\log(|v_{ij}| + 1)|} \frac{v_{ij}}{|v_{ij}|}. \quad (4)$$

One of the main properties of this normalization is to map zero in zero, one could see this property as preserving the sparsity pattern of A . In the Appendix B we show some examples of the combination of pooling and normalization for some matrices A and confront them with the relative sparsity pattern.

3.4. The neural network architecture

Once we have $\hat{V}$, we feed it in the neural network shown in Fig. 1, which we are now going to describe. We employ an ANN $\mathcal{F}$ to predict, as target the computational cost t and the square of the error committed by the ANN itself σ^2 . The insight behind σ is that, since the elapsed CPU time t is polluted with noise due to the measurements, in this way we are able to know when the ANN is confident on its own prediction.

The architecture of $\mathcal{F}$ is comprised by two components. The first one leverages a CNN to extract the relevant features from the matrix V . These features are flattened in an intermediate dense layer, where they are concatenated with the other features $p, \log_2(n_1)$ and θ and fed into a dense feed forward network, which predicts the computational cost t . On the output layer we clip the prediction of the normalized computational cost between zero and one and use a softplus activation function for the variance estimate.

We employ as inputs of the ANN the vector formed by the pooling of the matrix A $\hat{V}$, the normalized result of $\text{pooling}(A, m)$, where the size m is a hyperparameter that we tune (see Section 4.1.1), the FEM degree p , the logarithm of the size of A $\log_2(n_1)$ and the strong threshold θ . Namely,

$$\mathcal{F}(V, p, \log_2(n_1), \theta; \gamma) = (\tilde{t}, \tilde{\sigma})$$

and we optimize (the superscript i indicates the i -th sample of the dataset of size T):

$$\min_{\gamma} \frac{1}{T} \sum_{i=1}^T \text{MSE}(\tilde{t}^i, \tilde{t}^i) + \text{MSE}(\tilde{\sigma}^i, (\tilde{t}^i - \tilde{t}^i)^2). \quad (5)$$

Eq. (5) could benefit from introducing weights for the individual loss terms, however, the order of magnitude of the two terms is similar, so we prefer to avoid adding an additional hyperparameter. Let us remark that p is not needed as input of the neural network. Indeed, the information about p is embedded into the matrix A : empirical results show that if we add p as input of the network, we need a lower number of epochs to reach the same loss, but it is still possible to effectively train the network without p . Hence, our algorithm is not limited to problems stemming from FE discretizations.

Given the matrix A associated to the linear system we want to solve, then our algorithm prescribes the default literature value of the strong threshold θ every time the weighted average variance $\hat{\sigma}$ of the map $A \mapsto \theta$,

Algorithm 4 ANN-enhanced AMG $\mathbf{u} = \text{ANN_AMG}(\mathbf{u}, \mathbf{A}, \mathbf{f}, \{(B_1^{(j)}), B_2^{(j)}\}_{j=k}^M, v_1, v_2, N_{\max}, \text{tol})$.

```

1:  $\mathbf{V} \leftarrow \text{pooling}(\mathbf{A}, m)$  (Algorithm 3)
2: normalize  $\mathbf{V}$  by means of Eq. (4), obtaining  $\hat{\mathbf{V}}$ 
3:  $(\tilde{r}^j, \tilde{\sigma}^j) \leftarrow \mathcal{F}(\hat{\mathbf{V}}, p, \log_2(n_1), \frac{j}{200}; \gamma)$ ,  $j = 10, 11, \dots, 190$ 
4:  $\hat{\sigma} \leftarrow \frac{1}{181} \sum_{j=10}^{190} (1 - \tilde{r}^j) \tilde{\sigma}^j$ 
5:  $\theta^* \leftarrow \argmin_{\theta} \frac{j}{200}$  if  $\hat{\sigma} > \bar{\sigma}$  else 0.5
6:  $\mathbf{u} \leftarrow \text{AMG}(\mathbf{u}, \mathbf{A}, \mathbf{f}, \theta^*, \{(B_1^{(j)}), B_2^{(j)}\}_{j=k}^M, v_1, v_2, N_{\max}, \text{tol})$  (Algorithm 2)

```

$$\hat{\sigma} = \frac{1}{181} \sum_{j=10}^{190} (1 - \tilde{r}^j) \tilde{\sigma}^j, \text{ where } (\tilde{r}^j, \tilde{\sigma}^j) = \mathcal{F}(\mathbf{V}, p, \log_2(n_1), \frac{j}{200}; \gamma) \quad (6)$$

is greater than a certain threshold $\bar{\sigma}$. In Eq. (6) we consider $\theta = \frac{j}{200} \in [0.05, 0.95]$ because we found that smaller or larger values of θ do not influence in a significant manner the performance of the AMG. This weighted average ensures that the variance of the prediction is more relevant if closer to the (expected) minima. The value $\bar{\sigma}$ is calibrated offline on the validation dataset once after the training of the ANN. For each matrix $\mathbf{A}^i$ in the dataset we compute the relative error indicator $\hat{\sigma}^i$. Then we propose to choose $\bar{\sigma}$ as the ordinate of the elbow point of the sorted errors indicators $\hat{\sigma}^i$. On the other hand, if $\hat{\sigma} < \bar{\sigma}$, the algorithm prescribes as θ the value θ^* found by solving the optimization problem

$$\theta^* = \argmin_{\theta \in (0,1]} (\mathcal{F}(\mathbf{V}, p, \log_2(n_1), \theta; \gamma))_1. \quad (7)$$

However, we empirically found that by solving the discretization of the above with two hundred linearly spaced points, we have an estimate of θ^* that, for our aim, is indistinguishable from the solution of the continuous problem with gradient descent. There are three reasons to use this approach instead of predicting directly θ^* . (i) We can quantify the expected improvement with respect to using the standard literature value of θ . (ii) We do not have to solve a new optimization problem just to add one sample to the dataset. (iii) From our experiments, this approach seems to be more robust with respect to directly predicting θ^* .

3.5. The AMG-ANN algorithm

We now have all the ingredients to describe the proposed algorithm in detail.

Offline phase First, collect, smooth and normalize the data following the procedure outlined in Section 3.1. Then train the neural network with the collected samples. More details on how this is practically done are given in Section 4.

Online phase It is detailed in the following, cf. also Algorithm 4: as prerequisites the algorithm requires the matrix $\mathbf{A}$ of the linear system to be solved and the polynomial degree p of the discretization.

- 1 Compress $\mathbf{A}$ by applying the pooling to obtain $\mathbf{V}$ (Section 3.2).
- 2 Normalize $\mathbf{V}$ to obtain $\hat{\mathbf{V}}$ (Section 3.3)
- 3-5 Get θ^* by finding the θ that fed in the ANN, together with $\hat{\mathbf{V}}$, p , $\log_2(n_1)$, minimizes the predicted computational cost (Section 3.4).
- 6 Use θ^* as parameter of the AMG method.

3.5.1. Evaluating the performance of the algorithm

A small loss is not a good indicator of the performance of our algorithm. Indeed, the choice of θ is subordinate to the map $\mathbf{A} \mapsto \theta^*$ defined by Eq. (7). With this in mind, we introduce the following quantities of interest. Let $\mathbf{A}$ be fixed, and let:

- t_{ANN} be the computational time of the AMG-ANN algorithm

- $t_{0.25}$ be the computational time of the AMG method for $\theta = 0.25$
- t_{MIN} be the computational time of the AMG method with

$$\theta^* = \argmin_{\theta \in \text{dataset for A}} t(\theta; \mathbf{A}).$$
- $P = 1 - \frac{t_{\text{ANN}}}{t_{0.25}}$ be the performance index of the AMG-ANN algorithm
- $P_{\text{MAX}} = 1 - \frac{t_{\text{MIN}}}{t_{0.25}}$ be the best performance of the AMG-ANN algorithm.

Moreover, we can compound the quantities over different $\mathbf{A}$ and define P_B as the percentage of cases where $P \geq 0$, P_m as the average of P and P_M as the median of P . The ratio P/P_{MAX} gives a measure of how well the ANN has learned the data.

4. Numerical results

One of the advantages of our algorithm is that it integrates seamlessly into pre-existing code. We carry out several numerical experiments using deal.II [78,79] for the construction of the linear system and we rely on BoomerAMG [80] of the library HYPRE [81] as the AMG solver. No changes are made to BoomerAMG, we use it as a black-box solver and only change the value of the strong threshold parameter θ , all the other parameters (choice of pre- and post-smoother B , number of pre- and post-smoothing cycles v , etc.) are left as default. We set the tolerance on the (relative) residual to 10^{-8} .

Concerning the training of the ANN, we employ a 20-20-60 split of the dataset into training-validation-test, respectively. The splitting is done among problems, this means that a certain matrix $\mathbf{A}$ appears just in one of the three datasets. This entails that when we evaluate the algorithm on the test (or validation) dataset, it is making predictions on problems it has never seen before. Notice also that the test dataset is much larger than the other two: this makes sense only when the dataset is large enough so that the neural network can generalize well. We have also tested partitions of the dataset with larger training data: while this, indeed, increases accuracy and the performance of our algorithm on the test dataset, we have found no significant improvements when testing on unseen geometries (cf. Section 4.1.5). We believe that generalization to problems that are “too different” (e.g. domains with different topologies or different PDEs models) is a very challenging task. In this regard, we have found that techniques such as transfer learning hold greater relevance and potential for enhancing generalization capabilities (cf. Section 4.3.1). Training has been performed using the Adam optimizer with default learning rate 10^{-3} and minibatch size equal to 32. Moreover, we found that applying a suitable learning rate schedule is key to accelerate the optimization. Namely, we employ a learning rate schedule that halves the learning rate with patience 15 epochs.

The convolutional part of the network is composed by one to three plain convolutional blocks each one ending with a max-pooling layer. For each test case, we tune the number of convolutional layers, the size of the filter, the number of filters the number of dense layers and the width of the dense layers.

4.1. Test Case 1: strongly heterogeneous diffusion problem, unstructured grids

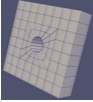
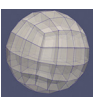
Let us consider the following parametric elliptic problem

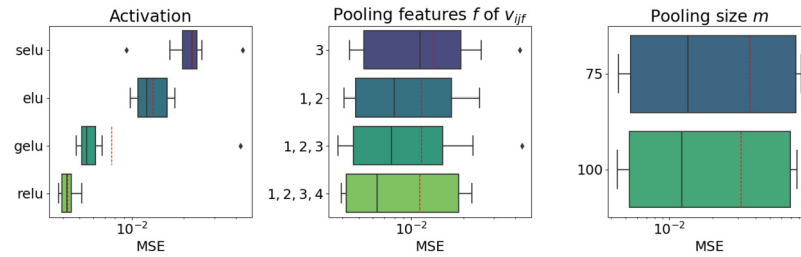
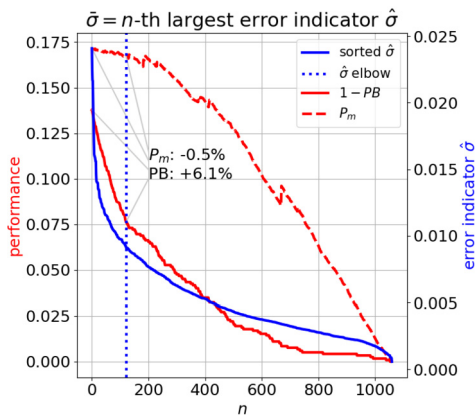
$$\begin{cases} -\text{div}(\mu \nabla u) = f, & \text{in } \Omega, \\ u = 0, & \text{on } \partial\Omega, \end{cases} \quad (8)$$

where $\mu \in L^\infty(\Omega)$ is a strongly heterogeneous piecewise positive constant and Ω varies among four geometries: a simplex, a plate with a hole, a ball and a cylinder. We consider in total eight different discretizations of the four domains and their nested refinements. We show the coarsest meshes in the first column of Table 1. The diffusion coefficient μ is a strongly heterogeneous piecewise constant that is conform-

Table 1

Test case 1: list of meshes used in the dataset.

Picture	Geometry	Description
	Simplex	Tetrahedron inscribed in the unit ball centered in (0, 0, 0).
	Plate with hole	The $(-2, 2) \times (-2, 2) \times (-\frac{1}{2}, \frac{1}{2})$ plate with a centered hole of radius 0.4 along the z axis. Discretizations with two or eight slices in the z-direction are considered.
	Ball	Unit ball centered in zero.
	Balanced ball	A variant of the unit ball centered in zero that has a better balance between the size of the cells around the outer curved boundaries and the cell in the interior.
	Cylinder	Cylinder of unit radius and height two centered in zero. We consider three different discretizations with one, two or eight slices along its height.
	Holes	3 × 2 replica of the “Plate with hole” geometry on the x and y. Only test dataset.
	Torus	Torus of circle radius 2 and inner radius $\frac{1}{2}$. The axis of the torus is the y-axis. Only test dataset.

**Fig. 4.** Boxplots of the impact of the choice of the activation function (left), the pooling features (center) and the pooling size m (right), on the validation loss (MSE) of an ANN trained on a subsample of the dataset described in Section 4.1. The red line represents the mean.**Fig. 5.** Test case 1: performance accuracy trade-off of the AMG-ANN algorithm depending on the choice of $\hat{\sigma}$. The algorithm is evaluated on the validation set.

ing with the mesh. Namely, μ has a different value equal to 10^{ϵ_i} on the i -th cell of one of the coarsening of the mesh, where ϵ_i is randomly chosen between $[0, \epsilon_{MAX}]$ and $\epsilon_{MAX} = 1, 3, 10$. The problem is discretized by means of continuous FE of order $p = 1, 2, 3$. In order to test the stabil-

ity of the algorithm we also use four different DoFs (degrees of freedom) numbering. When applying the renumbering, the underlying sparsity graph of the matrix A and thus the optimal value of θ stays the same, but the pooling V changes, so the ANN should learn to be invariant with respect to these changes. Namely we employed: the default dealii numbering, the Cuthill-McKee renumbering method implemented by dealii and by the Boost library [82] and King renumbering. We solve each problem with 37 values of θ with equispaced values between 0.05 and 0.95. In this way, we build a dataset counting 5873 different configurations and thus containing 217301 samples.

4.1.1. Study of hyperparameters and pooling features

In this section we analyze the influence of some hyperparameters on the final loss when we employ a subset of the dataset. This is a preliminary analysis that we made for each test case in order to choose the best value of m (the size of the pooling V), which of the features of V are relevant (that is which of the four layers f of v_{ijf} should we employ as input of the CNN) and the activation function. Namely we performed a grid search over different network architectures. Fig. 4 shows the results. The ReLU activation function consistently reaches the smallest loss, hence we employ ReLU activation functions in conjunction with He initialization [83]. The ANN that uses all four the features of the

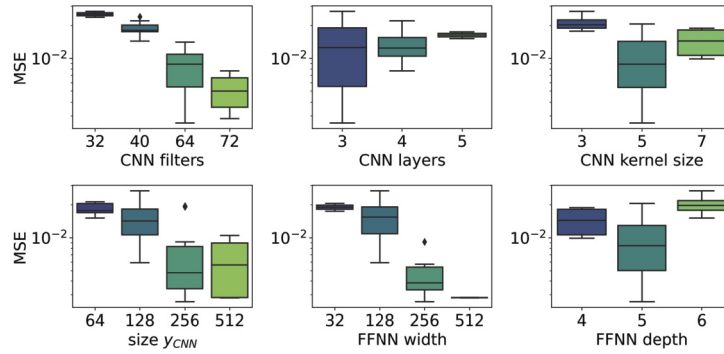


Fig. 6. Test case 1: Box plot of the validation loss values after 80 epochs depending on the choice of the following hyperparameters: the number of filters in the CNN layers, the number of CNN layers in each CNN block, the kernel size in the CNN layers, the size of the output of the CNN, the width and depth of the dense FFNN.

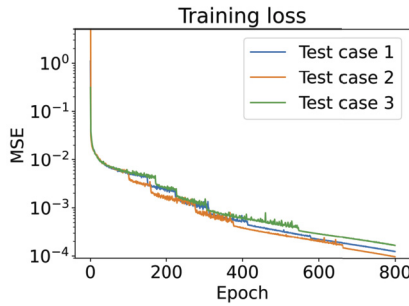


Fig. 7. History of the training loss of the fine-tuned ANN.

pooling has consistently the smallest loss. The pooling size $m = 75$ gives the best trade-off between accuracy and computational cost.

4.1.2. Calibration of the error threshold

We discuss how the choice of $\bar{\sigma}$ is made and how it impacts the performance of our algorithm. In Fig. 5 we plot the accuracy (PB) and average time reduction (P_m) when choosing as $\bar{\sigma}$ the n -th largest $\hat{\sigma}$ of the validation dataset. Notice there is an accuracy-performance trade-off, indeed, as $\bar{\sigma}$ tends to zero, PB tends to one but at the same time P_m tends to zero. Moreover, the fact that P_m is not strictly decreasing means that the error committed by the algorithm is not just due to noisy measurements. However, by using as $\bar{\sigma}$ the elbow of the ordered $\hat{\sigma}$ we have a gain of 6.1% in terms of accuracy meanwhile the mean performance decreases of only 0.5%.

4.1.3. Evaluation of the algorithm

Starting from the architecture we found in Section 4.1.1, we fine tune the number of filters, the kernel size of the CNN and the width and depth of the dense part. Our strategy is to use a grid-search on a fine space with a limited number of epochs and then we increase the number of epochs while discarding the combinations of hyperparameters that have the largest validation loss. This process is repeated until we have one combination of hyperparameters. Fig. 6 shows an example of the loss depending on the hyperparameters after 80 epochs. We report in Table 2 the final choice of hyperparameters. The ANN has a (test) loss of $2.89 \cdot 10^{-4}$, Fig. 7 shows the history of the training loss. The number of iterations needed reach convergence is between 6 and 155 with $\theta = 0.5$ and between 6 and 20 with $\theta = \theta^*$. The performance of our algorithm is evaluated by the indexes described in Section 3.5.1 on the test dataset (see Table 3) and by visualizing the scaling of the AMG with respect to the number of DoFs (see Fig. 8). Moreover, we remark that the scaling shown in the bottom row of Fig. 8, ideally, should be constant. Indeed, this is true when we apply the AMG to problems where $\max_{\Omega} \mu / \min_{\Omega} \mu \sim 1$, however in the cases that we consider it is not possible to achieve this. In any case, the AMG-ANN is able to achieve a much better scaling with respect to the not tuned AMG version. In

Table 2

Architecture of the ANN after the fine tune.

	Test case 1	Test case 2	Test case 3
CNN filters	64	48	64
CNN layers	3	3	3
CNN kernel size	6	5	5
y_{CNN} size	256	256	256
FNN width	512	256	512
FNN depth	512	256	512

particular, the algorithm has an accuracy of 92.51% and it produces in average a reduction of the computational cost of almost 16%. Since in this case the median on P_{MAX} is rather small (82%), there is still margin of improvement for the reduction of the computational cost by tuning the architecture of the ANN.

Fig. 9 shows all the predictions made by the trained ANN, notice that the largest points are the furthest from the bisector, thus the ANN is successfully predicting where it is inaccurate. In Fig. 10 we highlight some relevant predictions. Notice that the map $\theta \mapsto t$ exhibits many different complex patterns and that θ^* varies depending on the problem considered. Hence, it is not possible to find a fixed value θ^* that works well for all the problems. In Fig. 11, we showcase relevant cases of when the algorithm is not accurate, in particular, we categorize the errors of the algorithm into three classes (from left to right):

- Noisy measurements: the error is due to the measurement of t . We reduce this error by repeating the measurements and regularizing data. Moreover, $\bar{\sigma}$ is often a good indicator of a large error in these cases.
- Generalization error: this takes place when the network is not able to generalize beyond the training set. You can notice that in this case the prediction is completely wrong. However, a wrong prediction by the neural network of the curve $t(\theta)$ does not necessarily entail that the predicted θ^* is wrong too. Indeed, despite significant differences in the curves, their minima may be close. Therefore, we underline how the metric PB introduced in Section 3.5.1 is better suited to measure the accuracy of the neural network. In particular, the values of PB provided in Table 3, indicate that predictions are inaccurate in less than 10% of cases. Moreover, when employing the AMG-ANN algorithm with $\bar{\sigma} = \bar{\sigma}$, the cost increases by more than 15% in less than 1% of the cases.
- ANN error: in this category we classify the errors due to the approximation capacity of the ANN, the optimization error and the loss of information occurring during the pooling of A. These type of errors can be mitigated by hyperparameters tuning. However, there are some limitations, for instance, even if we would like m to be large, in order to lose the least amount of information possible, the cost of the evaluation of $\mathcal{F}$ depends quadratically on it, so it

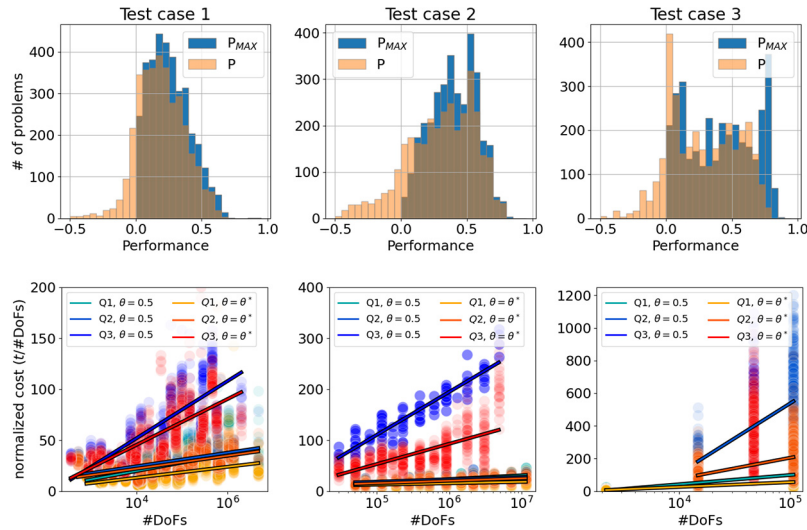


Fig. 8. (Top) Performance gain P of the AMG-ANN algorithm and maximum theoretical performance P_{MAX} for the three test cases. (Bottom) Scatter plot with linear regression of the DoF normalized computational cost of the AMG-ANN algorithm with respect to the number of DoFs and the polynomial degree p . In tones of red the AMG-ANN algorithm and in tones of blue the standard AMG method.

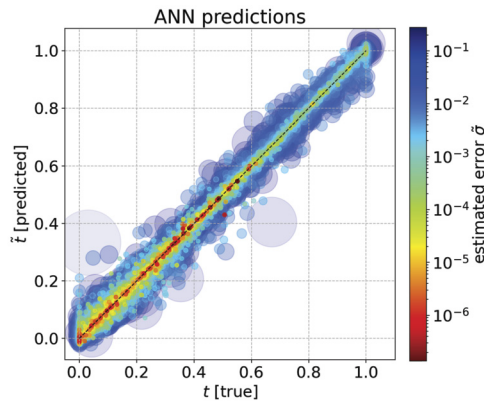


Fig. 9. Test case 1: Prediction of the ANN. Color and size are proportional to the estimated error $\bar{\sigma}$.

Table 3

Evaluation of the performance of the AMG-ANN algorithm for each test case depending on the choice of the threshold error $\bar{\sigma}$.

Test case	$\bar{\sigma}$	PB	P (avg/median)	P/P_{MAX} (median)
1: Unstructured	∞	86.58%	16.73%	17.90%
	$\bar{\sigma}$ elbow	92.51%	15.91%	15.46%
1: Holes	∞	80.82%	15.09%	18.02%
	$\bar{\sigma}$ elbow	86.01%	15.13%	17.76%
1: Torus	∞	63.88%	0.85%	5.69%
	$\bar{\sigma}$ elbow	65.08%	0.01%	3.30%
2: Structured	∞	87.97%	30.51%	32.14%
	$\bar{\sigma}$ elbow	93.03%	29.54%	19.42%
3: Elasticity	∞	83.72%	24.01%	21.40%
	$\bar{\sigma}$ elbow	90.71%	22.60%	18.54%
3: Pretrain	∞	85.76%	23.99%	23.79%
	$\bar{\sigma}$ elbow	92.39%	22.17%	22.31%

must stay small. Moreover, the error estimate $\bar{\sigma}$ is affected by these kind of errors itself, and it may not be of use.

4.1.4. Effect of the algorithm on sparsity

To investigate the impact of our algorithm on the sparsity of coarse grid matrices in AMG methods we analyze how the choice of θ made

Algorithm 5 Diffusion μ in a given point $\mathbf{x} \in \Omega = (-1, 1)^3$, having fixed the mode, size and $\epsilon \in \mathbb{R}^{\text{size}(\text{mode})}$. $\mu = \mu(\mathbf{x}; \text{mode}, \text{size}, \epsilon)$.

```

1:  $j \leftarrow 1$ 
2: for  $i = 0$  to mode do
3:    $j \leftarrow j + \lfloor ((\mathbf{x})_i + 1)\text{size}/2 \rfloor \text{size}^{i-1}$ 
4: end for
5:  $\mu \leftarrow 10^{(\epsilon)_j}$ 

```

by the ANN impacts the sparsity of the AMG levels. Fig. 14 shows that for large problems (number of DoF about 10^7) the sparsity of the coarse grid matrices remains manageable even as the problem size and number of AMG levels increase, independently of the choice of θ^* .

4.1.5. Evaluation on unseen domains

We tested the ANN-AMG algorithm by applying it to problems with a diffusion coefficient μ that was not present in the training dataset, but on a known geometry. To prove the generalization capability of the algorithm, we now test it using domains and diffusion coefficients it has never seen before. The two domains we employ are represented in the two last rows of Table 1: a replication of the plate with a hole in the x and y coordinates and a torus. We build the dataset exactly as done before, in total it counts 445 different problems. The performance on the “Holes” geometry is similar to a known geometry: $PB = 86\%$ and $P_M = 18\%$. Thus, we have shown that the algorithm is able to generalize also on domains with a different topology w.r.t. the ones in the training dataset. This result is linked to the fact that a (small) subset of the considered geometry (the plate with the hole) is present in the training dataset. Indeed, the results on the Torus are much worse: $PB = 64\%$ and $P_M = 6\%$. The algorithm is still making predictions that are significantly better than any prediction made “at random”, however this challenging mesh shows the limitation of our approach. We show the details about the performance in Table 3.

4.2. Test Case 2: strongly heterogeneous diffusion problem, structured grids

Let us consider the same elliptic problem (8) of before. However, here we have $\Omega = (-1, 1)^3$ and $\mu \in L^\infty(\Omega)$ is a strongly heterogeneous piecewise positive constant defined differently from before. Namely, μ is defined by means of Algorithm 5, where mode = 1, 2, 3 defines if the pattern is either made of slices, lines or is checkerboard like, size = 1, 2, ... defines how many times the pattern repeats, and the vector $\epsilon \in \mathbb{R}^{\text{size}(\text{mode})}$ defines the value of μ . Fig. 12 shows a represen-

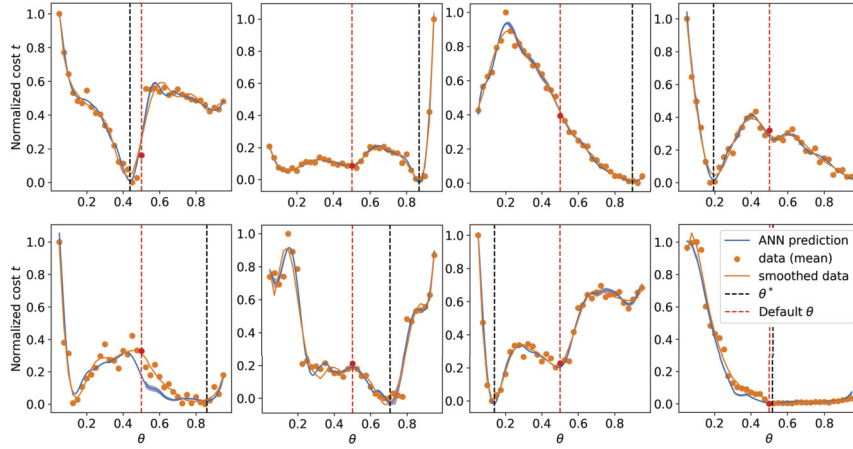


Fig. 10. Test case 1: Predictions made by the ANN for some exemplary values of the diffusion coefficient and domain shape. On the y-axis the normalized computational cost t , on the x-axis the value of the strong threshold parameter θ . The dots represent the raw measurements of the computational time.

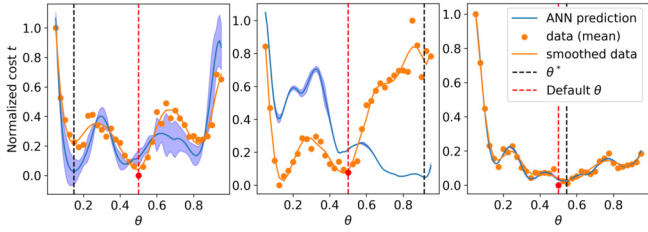


Fig. 11. Test case 1: three representative cases of predictions of the normalized computational cost t made by the ANN that leads to sub-optimal values of θ^* . The blue area is the ANN error estimate $\bar{\sigma}$.

tation of μ for $\text{size} = 5$ and $\text{mode} = 1, 2, 3$. The problem is discretized by means of continuous FE of order p on nested Cartesian meshes. The dataset is built by varying $p = 1, 2, 3$, $\text{mode} = 1, 2, 3$, $\text{size} = 2, 3, \dots, 10$, the mesh size h between $\frac{\sqrt[3]{2}}{\text{size}}$ and $\frac{\sqrt[3]{2}}{\text{size}2^{8-p}}$, and choosing at random the components of the vector ϵ between 0 and ϵ_{MAX} , where $\epsilon_{MAX} = 1, 3, 10$. We employ again four different DoFs numbering to enhance the dataset. We solve each problem with 37 values of θ equispaced between 0.05 and 0.95. The dataset we built counts 5471 different problems and thus contains 202427 samples.

We started by using an architecture with hyperparameters chosen as described in Section 4.1.1 and then tuned the number of filters and the size of the kernel of the CNN and the width and depth of the dense part of the ANN. We report in Table 2 the final choice of hyperparameters. We obtain an ANN with a (test) loss of $1.55 \cdot 10^{-4}$, Fig. 7 shows the history of the training loss. Details about its performance and its scaling are shown in Fig. 8 and Table 3. In particular, our algorithm has a 93% accuracy with an average reduction of the CPU time of almost 30%, and in half of the cases you can expect a reduction greater than 32%. The number of iterations needed reach convergence is between 7 and 18 with $\theta = 0.5$ and between 7 and 9 with $\theta = \theta^*$.

4.2.1. Study of a well-known family of problems

To better understand how the model problem we considered works and what does the ANN learn, we study a specific subset of the model problems defined in the previous section. Namely we consider a diffusion μ which is everywhere constant and equal to one apart from one cell of the mesh, where it is equal to $\mu_{MAX} = 10^\epsilon$ with $\epsilon = 2, 4, 8$. We stress that this family of problems was not present in the dataset we employed to train our ANN. Moreover, we define d to be the distance between the cell with the largest diffusion coefficient μ_{MAX} from the center of the domain. Fig. 13 (top-left) shows a graphical representation of the problem. First we analyze the conditioning of the matrix A . As ex-

pected it scales with respect to the mesh size h as h^{-2} and it is linearly proportional to the ratio between the maximum and minimum value of diffusion coefficient μ in the domain (i.e. is proportional to μ_{MAX}). The position of the cell does not affect the condition number. There is just a small reduction of it when the cell touches the boundaries: this is simply due to the Dirichlet boundary conditions, see Fig. 13 (bottom). We then test our algorithm. Fig. 13 (top-right) shows that the optimal value of θ is almost always in the same range: between 0.5 and 0.9. Moreover, we see that the ANN is able to capture the overall relation between the normalized computational cost t and the strong threshold. Hence, in all this cases, the predicted value θ^* by the AMG-ANN algorithm is near the true optimum. Finally, to gain some insight on the ANN we computed the feature maps extracted using the convolutional filters learned by the ANN. In Appendix C we show some of them: we can see that from a layer to another the CNN is enhancing the features that deems relevant.

4.3. Test Case 3: linear elasticity

We consider the following linear elasticity problem

$$\begin{cases} -\text{div}(\mathbb{C}\nabla_S \mathbf{u}) = \mathbf{f}, & \text{in } \Omega, \\ \mathbf{u} = \mathbf{0}, & \text{on } \partial\Omega, \end{cases} \quad (9)$$

where $\Omega = (-1, 1)^3$ and $\nabla_S \mathbf{u} = \frac{1}{2}(\nabla \mathbf{u} + \nabla \mathbf{u}^T)$ is the symmetric gradient and $\mathbb{C}$ is a rank-4 tensor that encodes the stress-strain relationship. Under the assumptions that the material under consideration is isotropic, by introducing the two Lamé coefficients $\lambda \in L^\infty(\Omega)$ and $\mu \in L^\infty(\Omega)$, the coefficient tensor reduces to

$$\mathbb{C}\nabla_S \mathbf{u} = 2\mu\nabla_S \mathbf{u} + \lambda\text{tr}(\nabla_S \mathbf{u})\mathbf{I}$$

where tr is the trace operator and $\mathbf{I}$ is the identity matrix. We can rewrite the Lamé parameters in terms of the Young's modulus $E > 0$ and the Poisson ratio $\nu \in (0, \frac{1}{2})$; we have $G = E/(1 + \nu)$ and $\beta = \nu/(1 - 2\nu)$. The problem has been discretized by means of FE of order $p = 1, 2, 3$ on a structured cartesian grid of diameter $h = \frac{\sqrt[3]{2}}{2}, \dots, \frac{\sqrt[3]{2}}{2^{4-p}}$. We fix the Poisson ratio $\nu = 0.29$ and choose a strongly heterogeneous Young modulus E . Namely E has the same pattern of the diffusion coefficient μ described in Section 4.2 and is such that size is even. We also employ four different DoFs numbering to enhance the dataset. We solve each problem with 37 values of θ equispaced between 0.05 and 0.95. In this way, we build a dataset counting 5873 different problems and thus containing 217301 samples.

As we have done for the previous test case, we start with an architecture found like described in Section 4.1.1 and then we fine tuned the

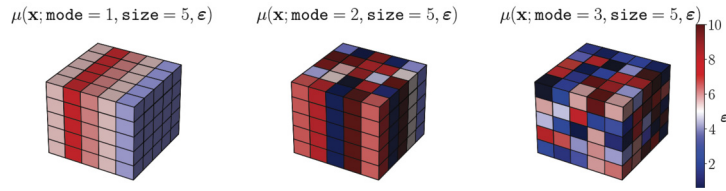


Fig. 12. Examples of the diffusion coefficient μ employed in Section 4.2 for some choices of the vector ϵ depending on the mode parameter (one to three from left to right) and fixed $\text{size}=5$.

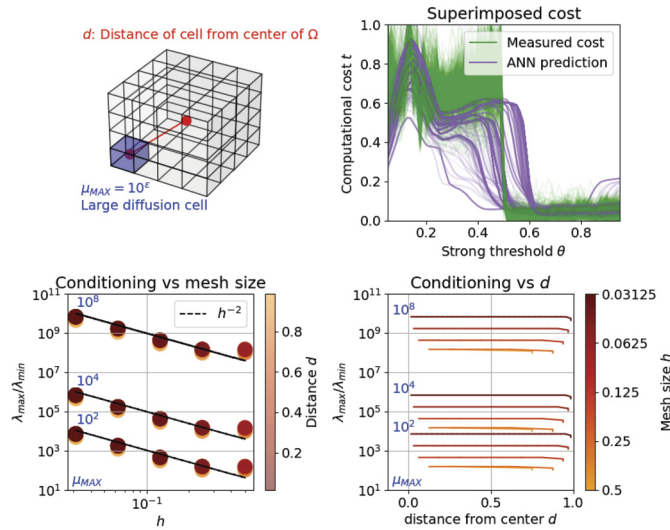


Fig. 13. Study of a special subset of problems of Test case 2. (Top-left) We consider a uniform diffusion $\mu = 1$ apart from one cell where $\mu = \mu_{MAX} = 10^6$ and $\epsilon = 2, 4, 8$. This cell has distance d from the origin. (Top-right) Superimposed normalized computational cost t needed to solve the system with the AMG method depending on the strong threshold parameter θ and relative predictions made by the ANN. (Bottom) Conditioning number of the matrix A depending on the distance d and the mesh size h .

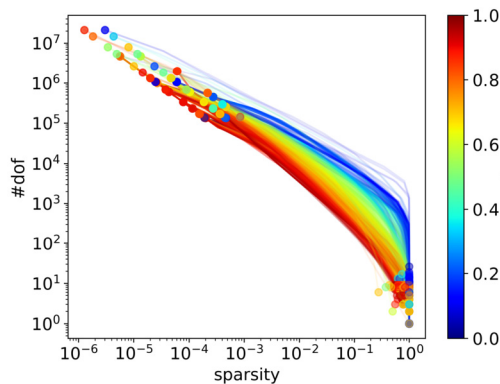


Fig. 14. Sparsity and size of the AMG levels depending on the choice of θ^* made by the AMG-ANN algorithm. First and last level are marked with a circle.

number of filters, the kernel size of the CNN and the width and depth of the dense part. The ANN has a (test) loss of $5.64 \cdot 10^{-4}$, Fig. 7 shows the history of the training loss. Evaluated on the test dataset, the algorithm has an accuracy of 90.71% and an average reduction of the computational cost of almost 23% in average. Details about its performance and its scaling are shown in Fig. 8 and Table 3. The number of iterations needed reach convergence is between 9 and 148 with $\theta = 0.5$ and between 8 and 87 with $\theta = \theta^*$. There is still margin of improvement concerning the reduction of the cost since P_{MAX} is relatively small,



Fig. 15. Relative performance improvement of the AMG method when applying our algorithm on Test case 3 before and after using transfer learning. The baseline is the ANN trained in Test case 2.

however it is possible to see a remarkable reduction of the scaling of the computational cost in the bottom row of Fig. 8.

4.3.1. Speeding up the training

We show that transfer learning and layer freezing can be used to speed up the training phase. We use the ANN we trained for Test case 2 as a pretrained network. Namely, we use it as the starting point of the training but we keep the weights of the convolutional block frozen. In this way we significantly reduce the cost of training the network since many parameters do not need the computation of the gradient. By caching the representation learnt by the CNN we can lower even more the cost of the training. Before the transfer learning we have $PB = 59.32\%$ and $P_M = 6.22\%$, which is slightly better than using $\theta = 0.5$. After the training, the loss is $3.97 \cdot 10^{-4}$, details about the performance are shown in Fig. 15 and Table 3. With a computational cost that is more than halved we have trained an ANN that performs better than the one trained from scratch.

Remark 1. A key aspect of the proposed AMG-ANN algorithm is how much data and variation in the data are required in order to obtain a good estimate of θ^* for your problem. Unfortunately, this is almost impossible to determine a-priori, since it is equivalent to determine how much an ANN generalizes, which is still an open problem of the field of DL. However, from the presented test cases we can conclude the following:

- The algorithm starts to struggle when making predictions on geometries Ω that has never seen before. It still proves effective if Ω is the union of geometries present in the dataset (see Section 4.1.5 for comparing the performance on “Holes” which is a replica of “Plate with hole”).
- The algorithm does not perform well when applied to problems with a different physics from the one in the training dataset (e.g., making prediction on linear elasticity problem using the dataset of Test case 2). However, transfer learning reduces significantly the training cost on new data. Indeed, we believe that simply increasing the size of the training data will not solve this problem. A careful design of a more complex dataset and an increase in

the number of parameters in the neural network is necessary to handle a variety of physics and geometries. However, this would significantly increase the computational cost of the algorithm. We think that techniques like transfer learning show more promise in addressing these challenges. Transfer learning allows us to use knowledge from pre-trained models without making the neural network unnecessarily large.

5. Conclusions

We extended the algorithm presented in [74] by adding features that allow it to encompass the complexity of three-dimensional problems. We discussed a pre-processing stage to ensure data quality, an augmented architecture of the ANN, and an enhanced version of the pooling for sparse matrices. Additionally, we demonstrated a method for accelerating training by implementing layer freezing techniques. This allows the algorithm to perform well also on a wide class of challenging problems coming from the FE discretization of elliptic PDEs with rough coefficients. We have provided numerical results for three different test cases concerning a Poisson problem with a strongly heterogeneous diffusion coefficient on (i) structured and (ii) unstructured grids and (iii) a linear elasticity problem with a strongly heterogeneous Young's modulus. Our algorithm performs better, other than using the standard literature value, in more than 90% of the cases and reduces significantly the computational cost (up to 30% on average). This is enabled by the fact that after the expensive training phase (which is done just once), the online cost of the algorithm is negligible.

The algorithm hinges upon the pooling operator to compress a large sparse matrix into a small tensor to feed into an ANN. Indeed, we have proved that the pooling operator reduces the computational cost and preserves relevant information needed to perform the complex regression task at hand. Moreover, our algorithm can be introduced with minimal changes into any existing code that uses an AMG solver.

In future works, we plan to generalize our algorithm by using one single ANN to make predictions on linear systems stemming from different underlying PDEs and being able to predict the optimal value of other parameters of the AMG method such as the type of smoother and the number of smoothing steps. We also plan to try more advanced computer vision models, such as transformers [61]. Finally, we aim to further investigate the properties of the pooling operator applied to sparse matrices and gain a better understanding of what the CNN learns.

CRedit authorship contribution statement

Matteo Caldana: Conceptualization, Data curation, Formal analysis, Investigation, Methodology, Software, Validation, Visualization, Writing – original draft. **Paola F. Antonietti:** Conceptualization, Funding acquisition, Methodology, Project administration, Supervision, Writing – review & editing. **Luca Dede':** Conceptualization, Funding acquisition, Methodology, Project administration, Supervision, Writing – review & editing.

Data availability

Data will be made available on request. The code is publicly available at <https://github.com/MatteoCaldana/AMG-ANN>.

Appendix A. Basic concepts of deep learning

A deep learning regression model is a function $\mathcal{F} : \mathbb{R}^{N_{IN}} \rightarrow \mathbb{R}^{N_{OUT}}$ that maps an input $\mathbf{x}$ to an output $\tilde{\mathbf{y}}$ and depends on a vector of parameters $\boldsymbol{\gamma}$. The parameters $\boldsymbol{\gamma}$ are chosen by minimizing the error (loss) evaluated on a training dataset of known input-output couples $\{(\mathbf{x}^i, \mathbf{y}^i)\}_{i=1}^T$, namely

$$\min_{\boldsymbol{\gamma}} \frac{1}{T} \sum_{i=1}^T \mathcal{L}(\mathbf{y}^i, \mathcal{F}(\mathbf{x}^i, \boldsymbol{\gamma}))$$

where $\mathcal{L}$ is the loss function. Namely, we employ the mean squared error (MSE). The optimization is performed by means of gradient descent updates that uses a mini-batch of the training dataset to compute $\nabla_{\boldsymbol{\gamma}} \mathcal{F}$ with automatic differentiation.

Feed-forward neural networks Let $\mathbf{y}^{(0)} = \mathbf{x}$ and $\mathbf{y}^{(L)} = \tilde{\mathbf{y}}$, a dense feed-forward neural network (FFNN) of depth L is the composition of L functions called layers defined as

$$\mathbf{y}^{(l)} = h^{(l)}(\mathbf{W}^{(l)} \mathbf{y}^{(l-1)} + \mathbf{b}^{(l)}), \quad l = 1, \dots, L$$

where $\mathbf{W}^{(l)} \in \mathbb{R}^{N_l \times N_{l-1}}$ (weights) and $\mathbf{b}^{(l)} \in \mathbb{R}^{N_l}$ (biases) are the parameters $\boldsymbol{\gamma}$, and $h^{(l)}$ is a scalar non-linear activation function applied component-wise.

Convolutional neural networks Convolutional neural networks (CNNs) are neural networks that use convolution instead of matrix multiplication in at least one of their layers. They have great success in computer vision tasks thanks to their ability to exploit the structured data format of images. Indeed, convolution layers enjoy three properties: shared parameters, that is, each parameter is tied to multiple component of the input; sparse interactions, that is, each component of $\mathbf{y}^{(l)}$ depends only on a subset of the components of $\mathbf{y}^{(l-1)}$, (this also entails a lower number of parameters and thus a greater efficiency); equivariance to translation, that is the application of a translation and a convolution can be interchanged to obtain the same result. Considering a three-dimensional input $\mathbf{Y}^{(0)} = \mathbf{X} \in \mathbb{R}^{N_1^{(0)} \times N_2^{(0)} \times N_3^{(0)}}$ and a bank of $K^{(l)} > 0$ convolutional filters in layer l denoted as $\mathbf{W}_k^{(l)} \in \mathbb{R}^{n_1^{(l)} \times n_2^{(l)} \times n_3^{(l)}}$, $l = 1, \dots, L$ and $k = 1, \dots, K^{(l)}$ the k -th feature map is computed as

$$\mathbf{Y}_k^{(l)} = h^{(l)}(\mathbf{W}_k^{(l)} * \mathbf{Y}_k^{(l-1)} + \mathbf{b}_k^{(l)})$$

where $\mathbf{Y}^{(l)} \in \mathbb{R}^{N_1^{(l)} \times N_2^{(l)} \times N_3^{(l)}}$ with $N_1^{(l)}$ and $N_2^{(l)}$ depends on $n_1^{(l)}$ and $n_2^{(l)}$ respectively, the padding and the striding strategies, and $N_3^{(l)} = K^{(l)}$. Here, $*$ denotes the two-dimensional convolution, often implemented as cross-correlation in the neural network libraries [84]:

$$(\mathbf{W}_k^{(l)} * \mathbf{Y}_k^{(l-1)})_{ij} = \sum_{m_1=1}^{n_1^{(l)}} \sum_{m_2=1}^{n_2^{(l)}} \mathbf{Y}_{i+m_1, j+m_2, k}^{(l-1)} \mathbf{W}_{m_1 m_2 k}^{(l)}$$

The last ingredient of a CNNs layer is the pooling operator. After several parallel convolutions and a non-linear activation, we replace the output of the layer at a certain location with a summary statistic of the nearby outputs. A popular option is the max-pooling [59], which reports the maximum value over a rectangular neighborhood. Namely, the max-pooling of window m , given in input $\mathbf{X} \in \mathbb{R}^{n \times n}$, outputs $\mathbf{Y} \in \mathbb{R}^{n/m \times n/m}$ such that

$$y_{ij} = \max_{i_1, j_1=1, \dots, m} x_{mi+i_1, mj+j_1}.$$

This definition can be extended to the case of n not divisible by m by padding $\mathbf{X}$ with a suitable amount of zeros. The aim of the pooling operation is to reduce the computational cost by limiting the number of parameters in subsequent layers, learn hierarchical representations of features, where lower layers capture low-level details, and higher layers capture more abstract and complex features, and adding invariance to small translations.

Appendix B. Pooling visualization

See Fig. B.16.

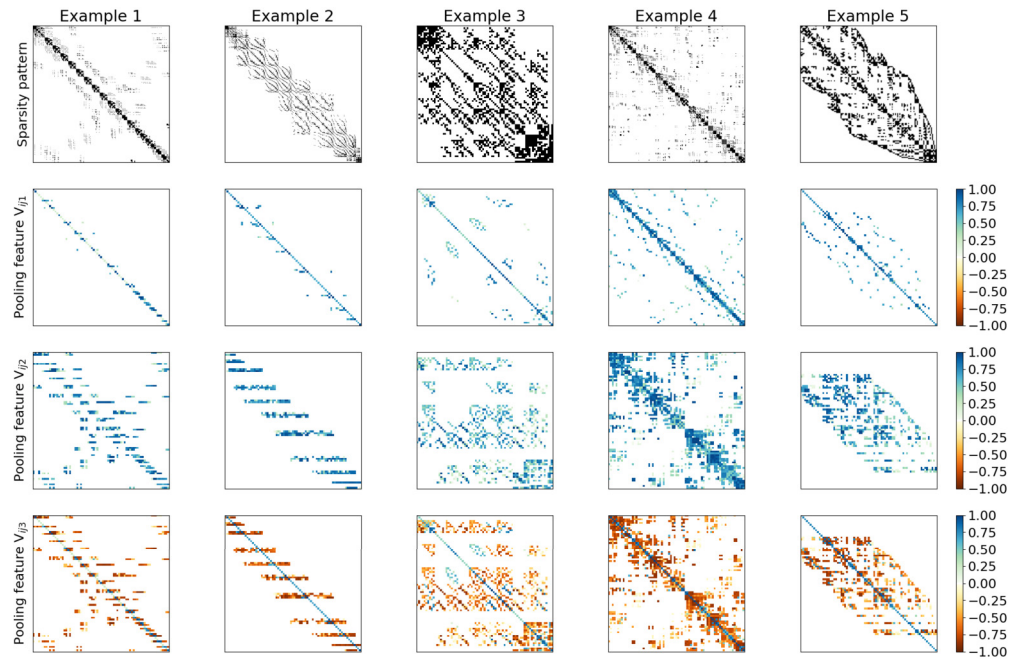


Fig. B.16. Visual representation of the normalized pooling results $\hat{V}$ that is fed into the ANN for some exemplary problems taken from the dataset described in Section 4.1.

Appendix C. CNN feature maps visualization

See Fig. C.17.

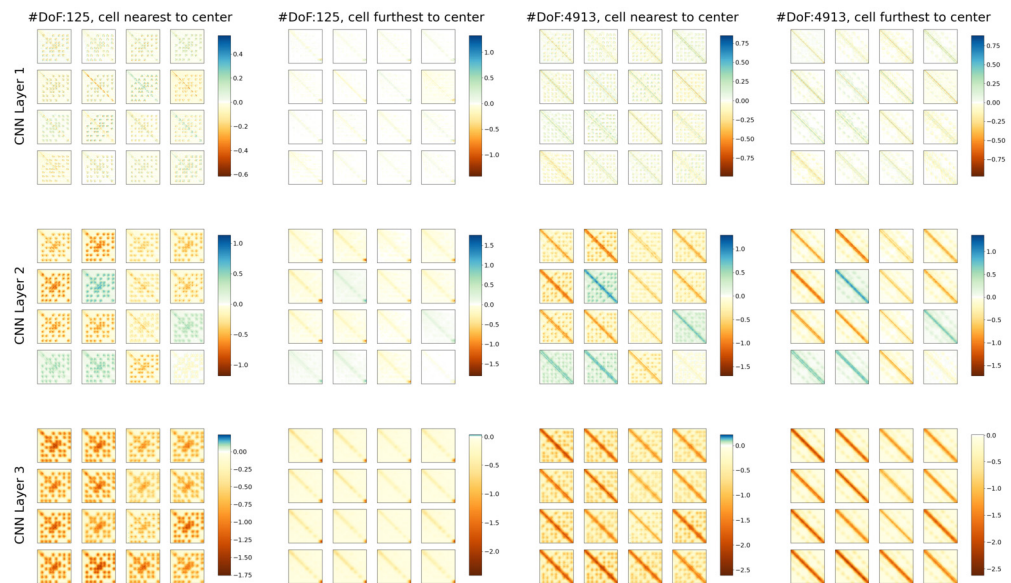


Fig. C.17. Example of feature maps of the CNN layers for the problems described in Section 4.2.1.

References

- [1] P. Wesseling, An Introduction to Multigrid Methods, An Introduction to Multigrid Methods, R.T. Edwards, 2004.
- [2] J.H. Bramble, Multigrid Methods, Chapman and Hall/CRC, 2019.
- [3] P.F. Antonietti, P. Houston, X. Hu, M. Sarti, M. Verani, Multigrid algorithms for hp-version interior penalty discontinuous Galerkin methods on polygonal and polyhedral meshes, *Calcolo* 54 (2017) 1169–1198.
- [4] P.F. Antonietti, G. Pennesi, V-cycle multigrid algorithms for discontinuous Galerkin methods on non-nested polytopic meshes, *J. Sci. Comput.* 78 (2019) 625–652.
- [5] P.F. Antonietti, L. Mascotto, M. Verani, A multigrid algorithm for the p-version of the virtual element method, *Modél. Math. Anal. Numér.* 52 (2018) 337–364.
- [6] P.F. Antonietti, S. Berrone, M. Busetto, M. Verani, Agglomeration-based geometric multigrid schemes for the virtual element method, *SIAM J. Numer. Anal.* 61 (2023) 223–249.
- [7] A.H. Baker, R.D. Falgout, T.V. Kolev, U.M. Yang, Scaling hypre's multigrid solvers to 100,000 cores, in: *High-Performance Scientific Computing: Algorithms and Applications*, 2012, pp. 261–279.
- [8] A. Brandt, S. McCormick, J. Ruge, Algebraic multigrid (AMG) for automatic multigrid solution with application to geodetic computations, in: *National Geodetic Survey and Air Force Office of Scientific Research and National Science Foundation*, 1983.
- [9] A. Brandt, Algebraic multigrid (AMG) for sparse matrix equations, in: *Sparsity and Its Applications*, 1984, pp. 257–284.
- [10] A. Brandt, Algebraic multigrid theory: the symmetric case, *Appl. Math. Comput.* 19 (1986) 23–56.
- [11] J.W. Ruge, K. Stüben, Algebraic Multigrid, in: *Multigrid Methods*, SIAM, 1987, pp. 73–130.
- [12] W.L. Briggs, V.E. Henson, S.F. McCormick, A Multigrid Tutorial, SIAM, 2000.
- [13] U. Trottenberg, C.W. Oosterlee, A. Schuller, *Multigrid*, Elsevier, 2000.
- [14] R.D. Falgout, An introduction to Algebraic Multigrid, Technical report, Lawrence Livermore National Lab. (LLNL), Livermore, CA (United States), 2006.
- [15] P.S. Vassilevski, Multilevel Block Factorization Preconditioners: Matrix-Based Analysis and Algorithms for Solving Finite Element Equations, Springer Science & Business Media, 2008.
- [16] P. Vaněk, J. Mandel, M. Brezina, Algebraic multigrid by smoothed aggregation for second and fourth order elliptic problems, *Computing* 56 (1996) 179–196.
- [17] T.F. Chan, J. Xu, L. Zikatanov, An agglomeration multigrid method for unstructured grids, *Contemp. Math.* 218 (1998) 67–81.
- [18] P. Vaněk, M. Brezina, J. Mandel, Convergence of algebraic multigrid based on smoothed aggregation, *Numer. Math.* 88 (2001) 559–579.
- [19] M. Brezina, A.J. Cleary, R.D. Falgout, V.E. Henson, J.E. Jones, T.A. Manteuffel, S.F. McCormick, J.W. Ruge, Algebraic multigrid based on element interpolation (AMGe), *SIAM J. Sci. Comput.* 22 (2001) 1570–1592.
- [20] J.E. Jones, P.S. Vassilevski, AMGe based on element agglomeration, *SIAM J. Sci. Comput.* 23 (2001) 109–133.
- [21] T. Chartier, R.D. Falgout, V. Henson, J. Jones, T. Manteuffel, S. McCormick, J. Ruge, P.S. Vassilevski, Spectral AMGe (ρ AMGe), *SIAM J. Sci. Comput.* 25 (2003) 1–26.
- [22] V.A. Dobrev, R.D. Lazarov, P.S. Vassilevski, L.T. Zikatanov, Two-level preconditioning of discontinuous Galerkin approximations of second-order elliptic equations, *Numer. Linear Algebra Appl.* 13 (2006) 753–770.
- [23] P. Bastian, M. Blatt, R. Scheichl, Algebraic multigrid for discontinuous Galerkin discretizations of heterogeneous elliptic problems, *Numer. Linear Algebra Appl.* 19 (2012) 367–388.
- [24] P.F. Antonietti, L. Melas, Algebraic multigrid schemes for high-order nodal discontinuous Galerkin methods, *SIAM J. Sci. Comput.* 42 (2020) A1147–A1173.
- [25] L. Botti, D.A. Di Pietro, p-Multilevel preconditioners for HHO discretizations of the Stokes equations with static condensation, *Commun. Appl. Math. Comput. Sci.* 4 (2022) 783–822.
- [26] D.A. Di Pietro, F. Hülsemann, P. Matalon, P. Mycek, U. Rüde, Algebraic multigrid preconditioner for statically condensed systems arising from lowest-order hybrid discretizations, *SIAM J. Sci. Comput.* (2023) S329–S350.
- [27] M. Raw, Robustness of coupled algebraic multigrid for the Navier-Stokes equations, in: *34th Aerospace Sciences Meeting and Exhibit*, 1996, p. 297.
- [28] J.M. Weiss, J.P. Maruszewski, W.A. Smith, Implicit solution of preconditioned Navier-Stokes equations using algebraic multigrid, *AIAA J.* 37 (1999) 29–36.
- [29] P.B. Bochev, C.J. Garasi, J.J. Hu, A.C. Robinson, R.S. Tuminaro, An improved algebraic multigrid method for solving Maxwell's equations, *SIAM J. Sci. Comput.* 25 (2003) 623–642.
- [30] T.V. Kolev, P.S. Vassilevski, Parallel auxiliary space AMG for H (curl) problems, *J. Comput. Math.* (2009) 604–623.
- [31] M. Griebel, D. Oeltz, M.A. Schweitzer, An algebraic multigrid method for linear elasticity, *SIAM J. Sci. Comput.* 25 (2003) 385–407.
- [32] N.A. Barnafi, L.F. Pavarino, S. Scacchi, A comparative study of scalable multilevel preconditioners for cardiac mechanics, *J. Comput. Phys.* (2023) 112421.
- [33] J.H. Adler, T.R. Benson, E.C. Cyr, S.P. MacLachlan, R.S. Tuminaro, Monolithic multigrid methods for two-dimensional resistive magnetohydrodynamics, *SIAM J. Sci. Comput.* 38 (2016) B1–B24.
- [34] J.A. White, N. Castelletto, S. Klevtsov, Q.M. Bui, D. Osei-Kuffuor, H.A. Tchelepi, A two-stage preconditioner for multiphase poromechanics in reservoir simulation, *Comput. Methods Appl. Mech. Eng.* 357 (2019) 112575.
- [35] A. Arrarás, F.J. Gaspar, L. Portero, C. Rodrigo, Multigrid solvers for multipoint flux approximations of the Darcy problem on rough quadrilateral grids, *Comput. Geosci.* 25 (2021) 715–730.
- [36] J.H. Bramble, J.E. Pasciak, J.P. Wang, J. Xu, Convergence estimates for multigrid algorithms without regularity assumptions, *Math. Comput.* 57 (1991) 23–45.
- [37] J. Xu, Iterative methods by space decomposition and subspace correction, *SIAM Rev.* 34 (1992) 581–613.
- [38] J. Xu, L. Zikatanov, The method of alternating projections and the method of subspace corrections in Hilbert space, *J. Am. Math. Soc.* 15 (2002) 573–597.
- [39] J. Xu, L. Zikatanov, On an energy minimizing basis for algebraic multigrid methods, *Comput. Vis. Sci.* 7 (2004) 121–127.
- [40] J. Xu, L. Zikatanov, Algebraic multigrid methods, *Acta Numer.* 26 (2017) 591–721.
- [41] S. McCormick, Multigrid methods for variational problems: general theory for the V-cycle, *SIAM J. Numer. Anal.* 22 (1985) 634–643.
- [42] L.T. Zikatanov, Two-sided bounds on the convergence rate of two-level methods, *Numer. Linear Algebra Appl.* 15 (2008) 439–454.
- [43] R.D. Falgout, P.S. Vassilevski, On generalizing the algebraic multigrid framework, *SIAM J. Numer. Anal.* 42 (2004) 1669–1693.
- [44] R.D. Falgout, P.S. Vassilevski, L.T. Zikatanov, On two-grid convergence estimates, *Numer. Linear Algebra Appl.* 12 (2005) 471–494.
- [45] S.P. MacLachlan, L.N. Olson, Theoretical bounds for algebraic multigrid performance: review and analysis, *Numer. Linear Algebra Appl.* 21 (2014) 194–220.
- [46] A. Quarteroni, A. Valli, *Numerical Approximation of Partial Differential Equations*, vol. 23, Springer Science & Business Media, 2008.
- [47] Y. LeCun, Y. Bengio, G. Hinton, Deep learning, *Nature* 521 (2015) 436–444.
- [48] G. Kutyniok, P. Petersen, M. Raslan, R. Schneider, A theoretical analysis of deep neural networks and parametric PDEs, *Constr. Approx.* 55 (2022) 73–125.
- [49] S. Mishra, A machine learning framework for data driven acceleration of computations of differential equations, *arXiv preprint*, arXiv:1807.09519, 2018.
- [50] R. Vinuesa, S.L. Brunton, Enhancing computational fluid dynamics with machine learning, *Nat. Comput. Sci.* 2 (2022) 358–366.
- [51] F. Regazzoni, L. Dede', A. Quarteroni, Machine learning for fast and reliable solution of time-dependent differential equations, *J. Comput. Phys.* 397 (2019) 108852.
- [52] S. Fresca, L. Dede', A. Manzoni, A comprehensive deep learning-based approach to reduced order modeling of nonlinear time-dependent parametrized PDEs, *J. Sci. Comput.* 87 (2021) 1–36.
- [53] F. Regazzoni, L. Dede', A. Quarteroni, Machine learning of multiscale active force generation models for the efficient simulation of cardiac electromechanics, *Comput. Methods Appl. Mech. Eng.* 370 (2020) 113268.
- [54] F. Regazzoni, M. Salvador, L. Dede', A. Quarteroni, A machine learning method for real-time numerical simulations of cardiac electromechanics, *Comput. Methods Appl. Mech. Eng.* 393 (2022) 114825.
- [55] S. Berrone, D. Oberto, An invariances-preserving vector basis neural network for the closure of Reynolds-averaged Navier–Stokes equations by the divergence of the Reynolds stress tensor, *Phys. Fluids* 34 (2022).
- [56] Y. LeCun, Y. Bengio, et al., Convolutional networks for images, speech, and time series, in: *The Handbook of Brain Theory and Neural Networks*, vol. 3361, 1995, p. 1995.
- [57] J. Hu, L. Shen, G. Sun, Squeeze-and-excitation networks, in: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 7132–7141.
- [58] F.N. Iandola, S. Han, M.W. Moskewicz, K. Ashraf, W.J. Dally, K. Keutzer, SqueezeNet: AlexNet-level accuracy with 50x fewer parameters and < 0.5 MB model size, *arXiv preprint*, arXiv:1602.07360, 2016.
- [59] A. Krizhevsky, I. Sutskever, G.E. Hinton, Imagenet classification with deep convolutional neural networks, *Commun. ACM* 60 (2017) 84–90.
- [60] J. Gu, Z. Wang, J. Kuen, L. Ma, A. Shahroudy, B. Shuai, T. Liu, X. Wang, G. Wang, J. Cai, et al., Recent advances in convolutional neural networks, *Pattern Recognit.* 77 (2018) 354–377.
- [61] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al., An image is worth 16x16 words: transformers for image recognition at scale, *arXiv preprint*, arXiv:2010.11929, 2020.
- [62] S. Bhatnagar, Y. Afshar, S. Pan, K. Duraisamy, S. Kaushik, Prediction of aerodynamic flow fields using convolutional neural networks, *Comput. Mech.* 64 (2019) 525–545.
- [63] M. Eichinger, A. Heinlein, A. Klawonn, Stationary flow predictions using convolutional neural networks, in: *Numerical Mathematics and Advanced Applications ENUMATH 2019: European Conference, Egmond aan Zee, the Netherlands, September 30–October 4*, Springer, 2020, pp. 541–549.
- [64] D. Greenfeld, M. Galun, R. Basri, I. Yavneh, R. Kimmel, Learning to optimize multigrid PDE solvers, in: *International Conference on Machine Learning*, PMLR, 2019, pp. 2415–2423.
- [65] A. Katrusa, T. Daulbaev, I. Oseledets, Black-box learning of multigrid parameters, *J. Comput. Appl. Math.* 368 (2020) 112524.
- [66] P. Antonietti, F. Dassi, E. Manuzzi, Machine learning based refinement strategies for polyhedral grids with applications to virtual element and polyhedral discontinuous Galerkin methods, *J. Comput. Phys.* 469 (2022) 111531.
- [67] P.F. Antonietti, E. Manuzzi, Refinement of polygonal grids using convolutional neural networks with applications to polygonal discontinuous Galerkin and virtual element methods, *J. Comput. Phys.* 452 (2022) 110900.
- [68] P. Antonietti, N. Farenga, E. Manuzzi, G. Martinelli, L. Saverio, Agglomeration of polygonal grids using graph neural networks with applications to multigrid solvers, *arXiv preprint*, arXiv:2210.17457, 2022.

- [69] I. Luz, M. Galun, H. Maron, R. Basri, I. Yavneh, Learning algebraic multigrid using graph neural networks, in: International Conference on Machine Learning, PMLR, 2020, pp. 6489–6499.
- [70] N.S. Moore, E. Cyr, C. Siefert, Learning an Algebraic Multigrid Interpolation Operator Using a Modified GraphNet Architecture, Technical Report, Sandia National Lab. (SNL-NM), Albuquerque, NM (United States), 2022.
- [71] A. Taghibakhshi, S. MacLachlan, L. Olson, M. West, Optimization-based algebraic multigrid coarsening using reinforcement learning, *Adv. Neural Inf. Process. Syst.* 34 (2021) 12129–12140.
- [72] A. Heinlein, A. Klawonn, M. Lanser, J. Weber, Machine learning in adaptive domain decomposition methods—predicting the geometric location of constraints, *SIAM J. Sci. Comput.* 41 (2019) A3887–A3912.
- [73] A. Heinlein, A. Klawonn, M. Lanser, J. Weber, Combining machine learning and domain decomposition methods for the solution of partial differential equations—a review, *GAMM-Mitt.* 44 (2021) e202100001.
- [74] P.F. Antonietti, M. Caldana, L. Dede', Accelerating algebraic multigrid methods via artificial neural networks, *Vietnam J. Math.* (2023) 1–36.
- [75] M. Brezina, R. Falgout, S. MacLachlan, T. Manteuffel, S. McCormick, J. Ruge, Adaptive algebraic multigrid, *SIAM J. Sci. Comput.* 27 (2006) 1261–1286.
- [76] A.J. Cleary, R.D. Falgout, V.E. Henson, J.E. Jones, Coarse-grid selection for parallel algebraic multigrid, in: Solving Irregularly Structured Problems in Parallel: 5th International Symposium, IRREGULAR'98 Berkeley, California, USA, August 9–11, 1998 Proceedings 5, Springer, 1998, pp. 104–115.
- [77] A. Savitzky, M.J. Golay, Smoothing and differentiation of data by simplified least squares procedures, *Anal. Chem.* 36 (1964) 1627–1639.
- [78] D. Arndt, W. Bangerth, D. Davydov, T. Heister, L. Heltai, M. Kronbichler, M. Maier, J.-P. Pelteret, B. Turcksin, D. Wells, The deal.II finite element library: design, features, and insights, *Comput. Math. Appl.* 81 (2021) 407–422, <https://doi.org/10.1016/j.camwa.2020.02.022>, <https://arxiv.org/abs/1910.13247>.
- [79] D. Arndt, W. Bangerth, B. Blais, T.C. Clevenger, M. Fehling, A.V. Grayver, T. Heister, L. Heltai, M. Kronbichler, M. Maier, et al., The deal.II library, version 9.2, *J. Numer. Math.* 28 (2020) 131–146.
- [80] U.M. Yang, et al., BoomerAMG: a parallel algebraic multigrid solver and preconditioner, *Appl. Numer. Math.* 41 (2002) 155–177.
- [81] R.D. Falgout, U.M. Yang, hypre: a library of high performance preconditioners, in: International Conference on Computational Science, Springer, 2002, pp. 632–641.
- [82] J.G. Siek, L.-Q. Lee, A. Lumsdaine, The Boost Graph Library: User Guide and Reference Manual, Pearson Education, 2001.
- [83] K. He, X. Zhang, S. Ren, J. Sun, Deep residual learning for image recognition, in: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition, 2016, pp. 770–778.
- [84] I. Goodfellow, Y. Bengio, A. Courville, Deep Learning, MIT Press, 2016, <http://www.deeplearningbook.org>.